

NOTE DE CADRAGE · DOCUMENT INTERNE

RADAR

Plateforme de veille concurrentielle
propulsée par un agent IA autonome

Document d'arbitrage technique interne du binôme. Fixe les zones de responsabilité et les décisions tranchées avant le démarrage du sprint 1. Le détail produit reste dans le PRD.

BINÔME
MODULE
ENCADRANT
INSTITUTION

Karamo Sylla · Bachirou Konaté
M244 · Veille Technologique
Pr. Younes Wadiai
ENSA Tétouan · Cycle Ingénieur BDIA

01 Synthèse du projet

Radar est un outil de veille concurrentielle propulsé par un agent IA autonome (OpenClaw) qui surveille quotidiennement les concurrents d'une Entreprise pour détecter changements de prix, lancements produits, recrutements, levées de fonds et campagnes marketing. L'utilisateur fournit nom et site web, le Deep Research enrichit automatiquement le profil entreprise, puis l'agent identifie et surveille les concurrents pertinents via cron quotidien à 6h.

Chaque cycle produit un rapport structuré (Synthèse, Collecte avec scores CRAAP, Analyse des Mouvements, Signaux Faibles, MAJ SWOT, Recommandations), des alertes scorées dans le dashboard, un digest email et la mise à jour des grilles SWOT quotidienne et PESTEL hebdomadaire.

Le MVP couvre 7 fonctionnalités, dont 3 critiques (Deep Research, Surveillance Concurrents, Dashboard) et 3 hautes (Email Digest, Analyses IA, Timeline 90 jours).
Cible : étudiants stratégie, dirigeants PME, consultants.

02 Répartition des rôles

Composant	Responsable	Stack envisagée
Front-end web (App Router, Server Components)	Karamo	Next.js (latest), TypeScript strict
Système de design et composants UI	Karamo	Tailwind CSS, Shadcn UI, Motion.dev
Visualisations dashboard (charts, timeline, KPI)	Karamo	Recharts
State management (global, serveur, URL)	Karamo	Zustand, TanStack Query, Nuqs
Formulaires et validation	Karamo	React Hook Form + Zod
Authentification et gestion utilisateurs	Karamo	Better Auth (email/password + Google OAuth)

Composant	Responsable	Stack envisagée
Modélisation et schéma BDD	Karamo	Prisma ORM + PostgreSQL 17 (Neon serverless)
UX/UI (parcours, wireframes, design system)	Karamo	Figma + tokens Tailwind
Templates email transactionnels et digest	Karamo	React Email + Resend
Export PDF côté client	Karamo	À choisir (react-pdf, Puppeteer côté back ?)
Agent IA autonome (browser control, planification)	Bachirou	OpenClaw (TypeScript)
Architecture multi-agent	Bachirou	Sous-agents spécialisés (Collecteur, Évaluateur, Analyste SWOT, Analyste PESTEL, Rédacteur)
Prompts agent et méthodologie M244 codée	Bachirou	Anthropic Claude (LLM principal)
Acquisition de données (scraping et extraction)	Bachirou	Playwright + extraction LLM
Calcul du score CRAAP par source	Bachirou	Heuristique ou LLM
Génération SWOT, PESTEL, signaux faibles	Bachirou	Claude + prompts structurés
Recoupement multi-sources et déduplication	Bachirou	Heuristique ou vectoriel
Orchestration des cycles de veille (cron 6h)	Bachirou	Cron + file de tâches
Cache et rate limiting	Bachirou	Upstash Redis
Conteneurisation OpenClaw + reverse proxy SSL	Bachirou	Docker + Caddy

Composant	Responsable	Stack envisagée
Contrat API Next.js ↔ OpenClaw	Bachirou (avec validation Karamo)	REST ou tRPC
Déploiement front	Karamo	Vercel (Next.js)
Déploiement back (OpenClaw)	Bachirou	VPS Docker (Caddy SSL)

03 Périmètre Karamo

Front-end · UX/UI · Base de données

- ▶ **Front-end.** Pages utilisateur (onboarding, dashboard, page concurrent, page rapport, timeline, paramètres).
- ▶ **UX/UI.** Wireframes Figma des 7 fonctionnalités MVP, design system aligné sur les tokens Tailwind.
- ▶ **Base de données.** Schéma Prisma (entités principales : User, Workspace, Company, Competitor, Source, Movement, Alert, Report, SWOTGrid, PESTELGrid, WeakSignal), indexation des requêtes critiques, rétention 90 jours.
- ▶ **Emails.** Templates React Email (digest quotidien et hebdomadaire, alertes critiques).
- ▶ **Points ouverts.** Streaming des rapports longs (SSE vs polling), versioning des grilles SWOT/PESTEL, stockage des snapshots HTML bruts.

04 Périmètre Bachirou

Back-end · Cerveau

- ▶ **Acquisition.** Scraping headless via OpenClaw + extraction LLM pour les contenus non structurés.
- ▶ **Traitement.** Pipeline collecte → extraction → CRAAP → classification par axe → recoupement multi-sources → scoring → rapport → MAJ SWOT/PESTEL.
- ▶ **Orchestration.** Cron quotidien 6h (surveillance), hebdomadaire (PESTEL), ad hoc (Deep Research). Retry, file persistante, gestion des coûts LLM.

- ▶ **API.** Contrat Next.js ↔ OpenClaw, authentification inter-services par token, webhooks de fin de cycle.

05 Décisions tranchées

Langage back-end : TypeScript full-stack. OpenClaw est natif TypeScript, ce qui élimine la friction d'un double écosystème et aligne tout le projet sur une seule stack.

Architecture multi-agent : sous-agents spécialisés. Un agent par étape du cycle de veille (Collecteur, Évaluateur de sources, Analyste SWOT, Analyste PESTEL, Rédacteur), ce qui rend le mapping pédagogique vers le cours immédiat et améliore la débuggabilité.

Conteneurisation : Docker. OpenClaw est conteneurisé en Docker, derrière Caddy en reverse proxy SSL.